# IRIS Dataset

## Decision Trees, Random Forests, Ensemble Methods

# Last week

- Classification
- Logistic Regression
- Performance Metrics for classification
- Classification using MNIST dataset (hand-drawn digits)
- Sklearn API

# This week

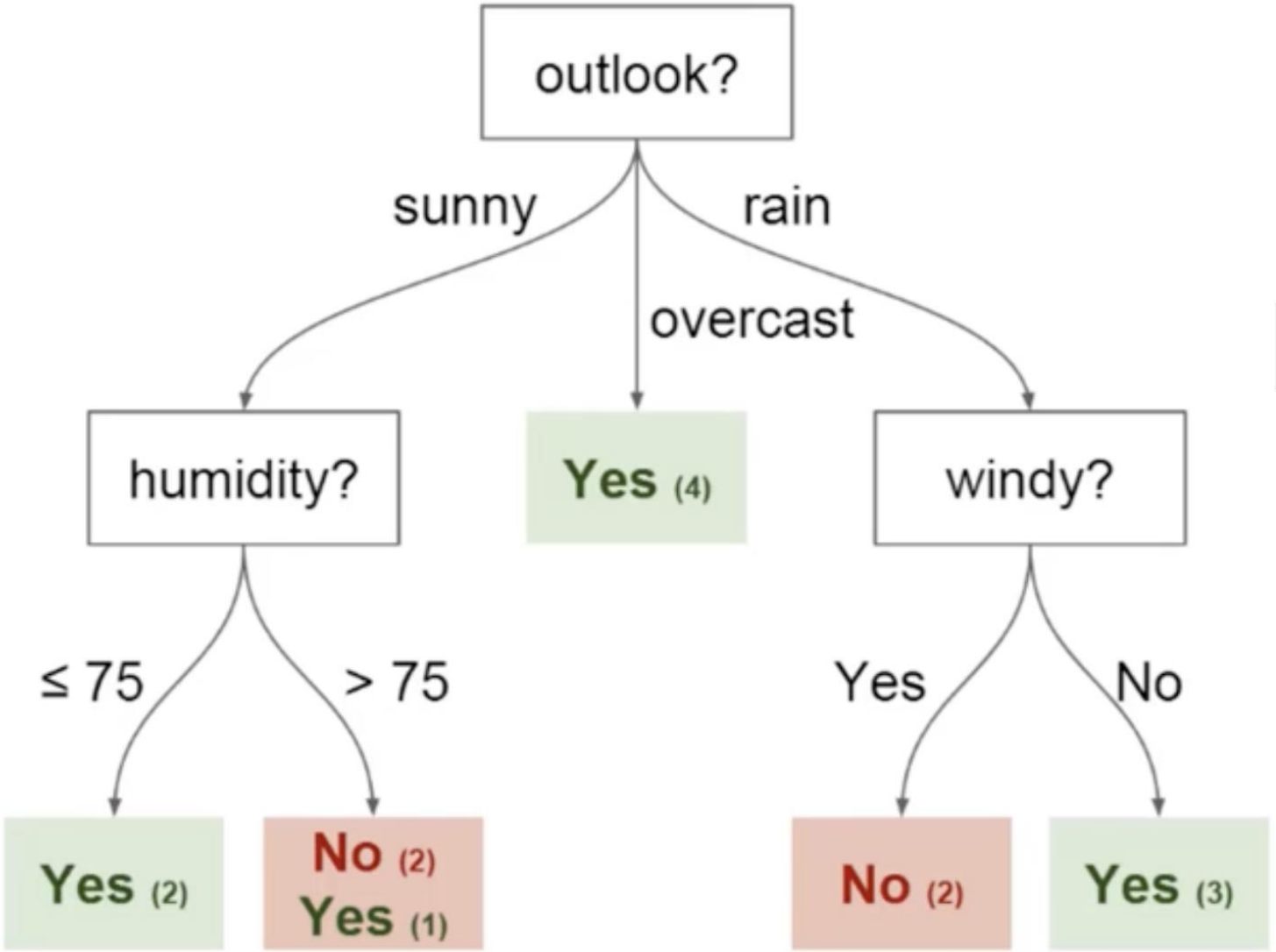
- Decision Trees
- Metrics: Gini Impurity, Entropy and Information Gain
- Ensemble Methods:
  - Random Forests (a bagging technique)
  - Boosting (gradients):
    - AdaBoost
    - Gradient Boosting
    - XGBoost
  - Voting and averaging

# Decision Trees

- Decision Trees can be organised around decisions. A question (condition) is asked (evaluated), and options (answers) are branches that may lead either to another ‘fork’ or a classification (category).
- The aim is to structure a tree so that it leads to accurate **classifications** based upon the logic that underpins them.
- Supervised Learning algorithm – trained to classify new data based upon patterns (decisions).

# Tree terminology

- Nodes
- Edges
- Root
- Leaves



# Example CSV

- Columns represent influential factors (conditions).
- Rows are values for each of these factors.
- The idea is to train the decision tree on this data.

| Temperature | Outlook | Humidity | Windy | Played? |
|---|---|---|---|---|
| Mild | Sunny | 80 | No | Yes |
| Hot | Sunny | 75 | Yes | **No** |
| Hot | Overcast | 77 | No | Yes |
| Cool | Rain | 70 | No | Yes |
| Cool | Overcast | 72 | Yes | Yes |
| Mild | Sunny | 77 | No | **No** |
| Cool | Sunny | 70 | No | Yes |
| Mild | Rain | 69 | No | Yes |
| Mild | Sunny | 65 | Yes | Yes |
| Mild | Overcast | 77 | Yes | Yes |
| Hot | Overcast | 74 | No | Yes |
| Mild | Rain | 77 | Yes | **No** |
| Cool | Rain | 73 | Yes | **No** |
| Mild | Rain | 78 | No | Yes |

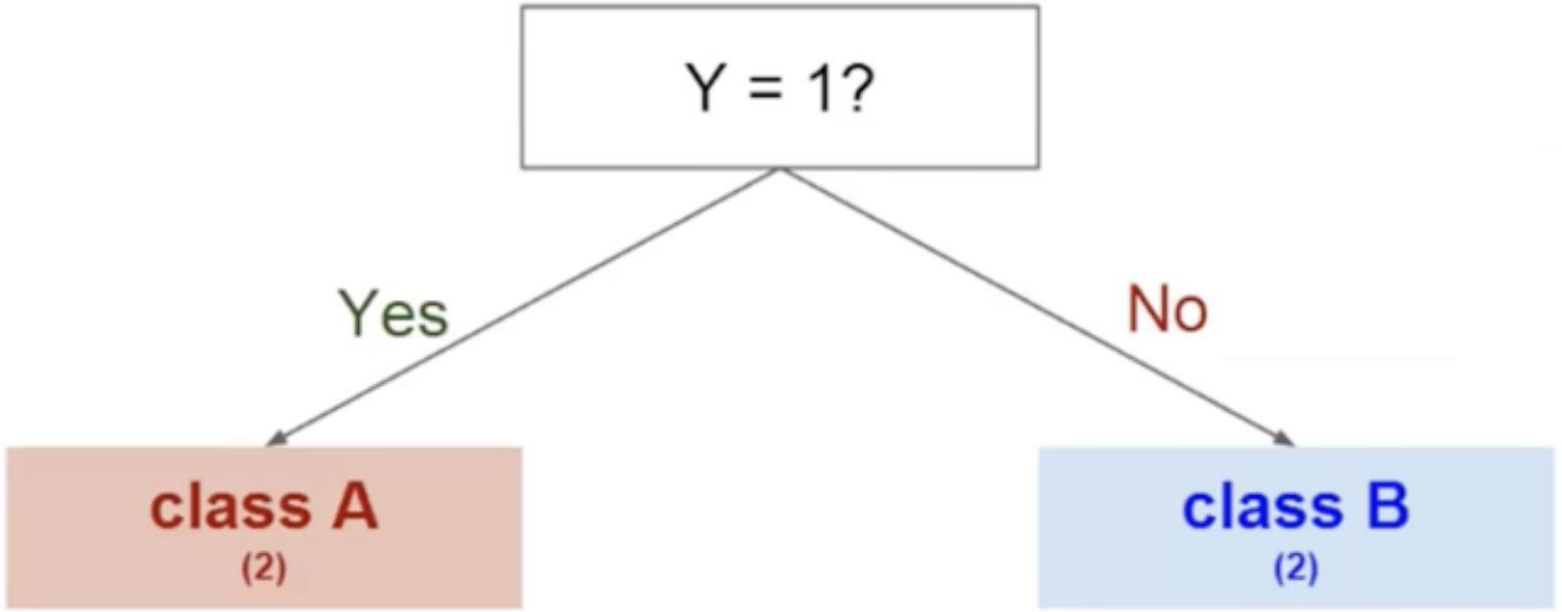
# Different labels

- Imaginary data with 3 features (X, Y and Z) with two possible classes.

| X | Y | Z | Class |
|---|---|---|---|
| 1 | 1 | 1 | A |
| 1 | 1 | 0 | A |
| 0 | 0 | 1 | B |
| 1 | 0 | 0 | B |

# Branching out

- Splitting on Y gives us a clear separation between classes



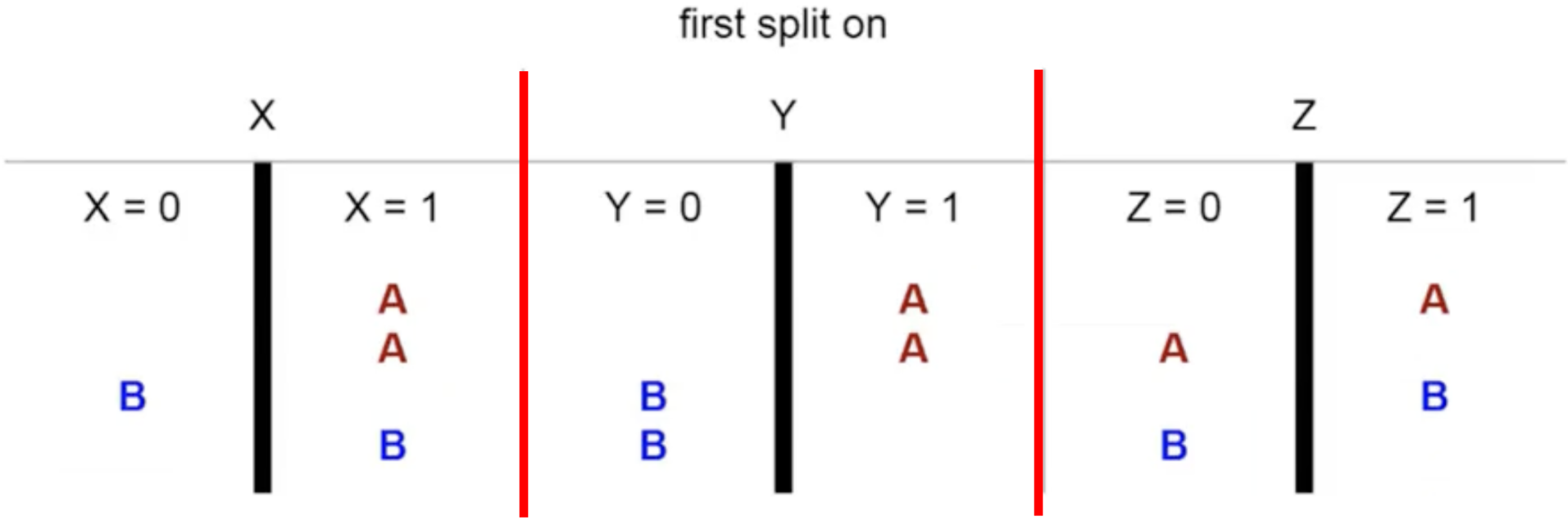
# Clear differentiation between classes

- Y = 1 for Class A, and Y = 0 for Class B (for this small dataset).

| X | Y | Z | Class |
|---|---|---|---|
| 1 | 1 | 1 | A |
| 1 | 1 | 0 | A |
| 0 | 0 | 1 | B |
| 1 | 0 | 0 | B |

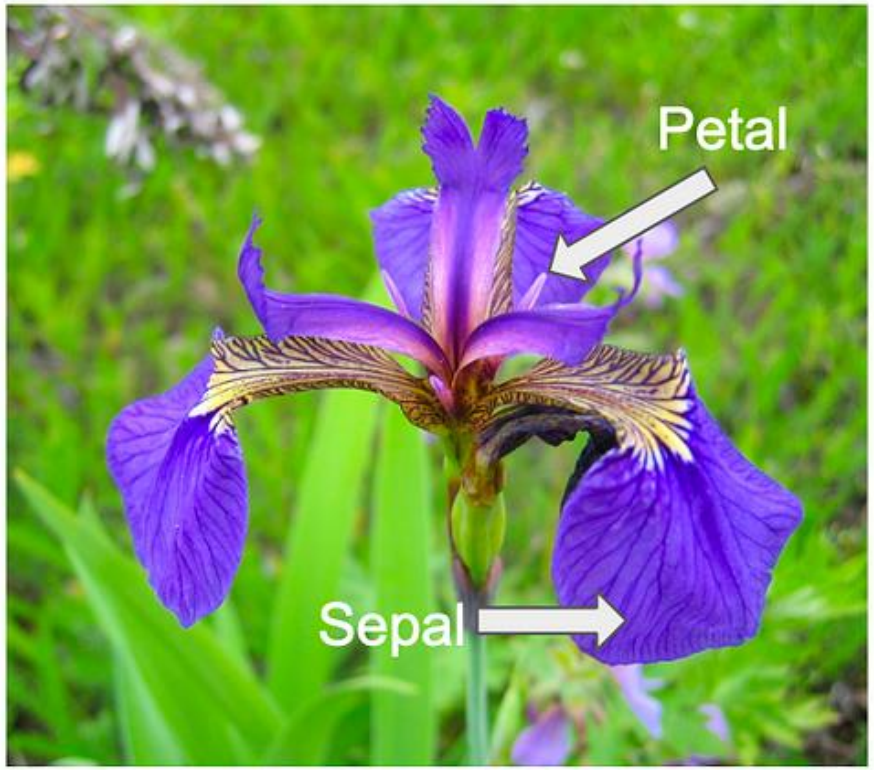
# Other combinations

- There are other combinations to ‘split’ by, but not all lead to accurate classification…

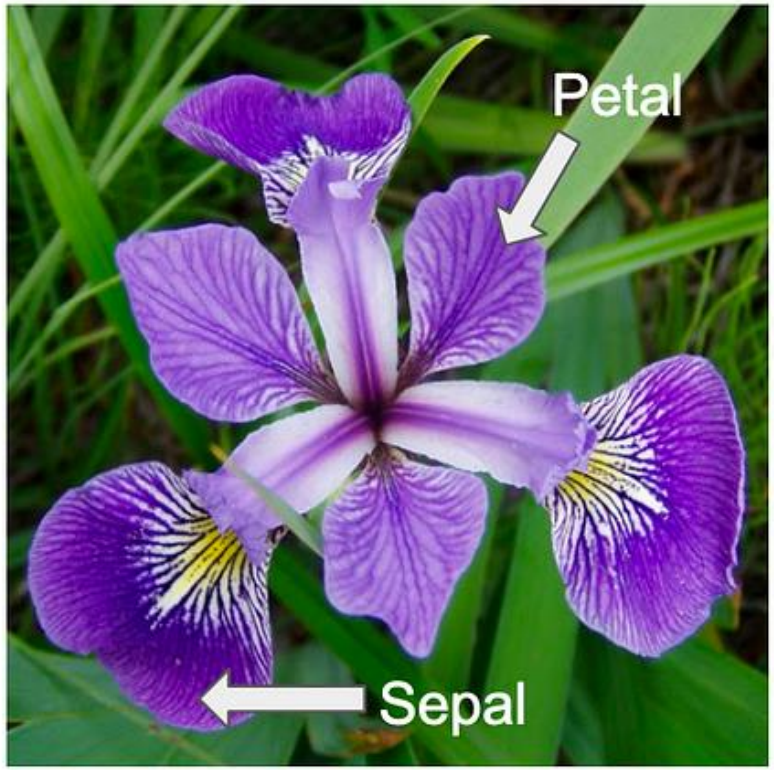


# Iris Dataset

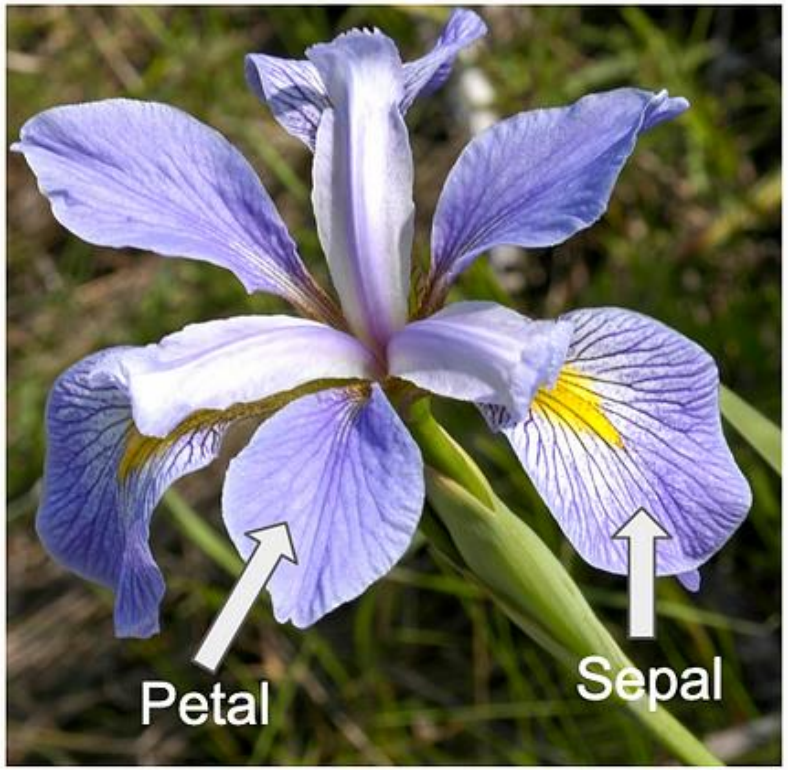
*Iris setosa*



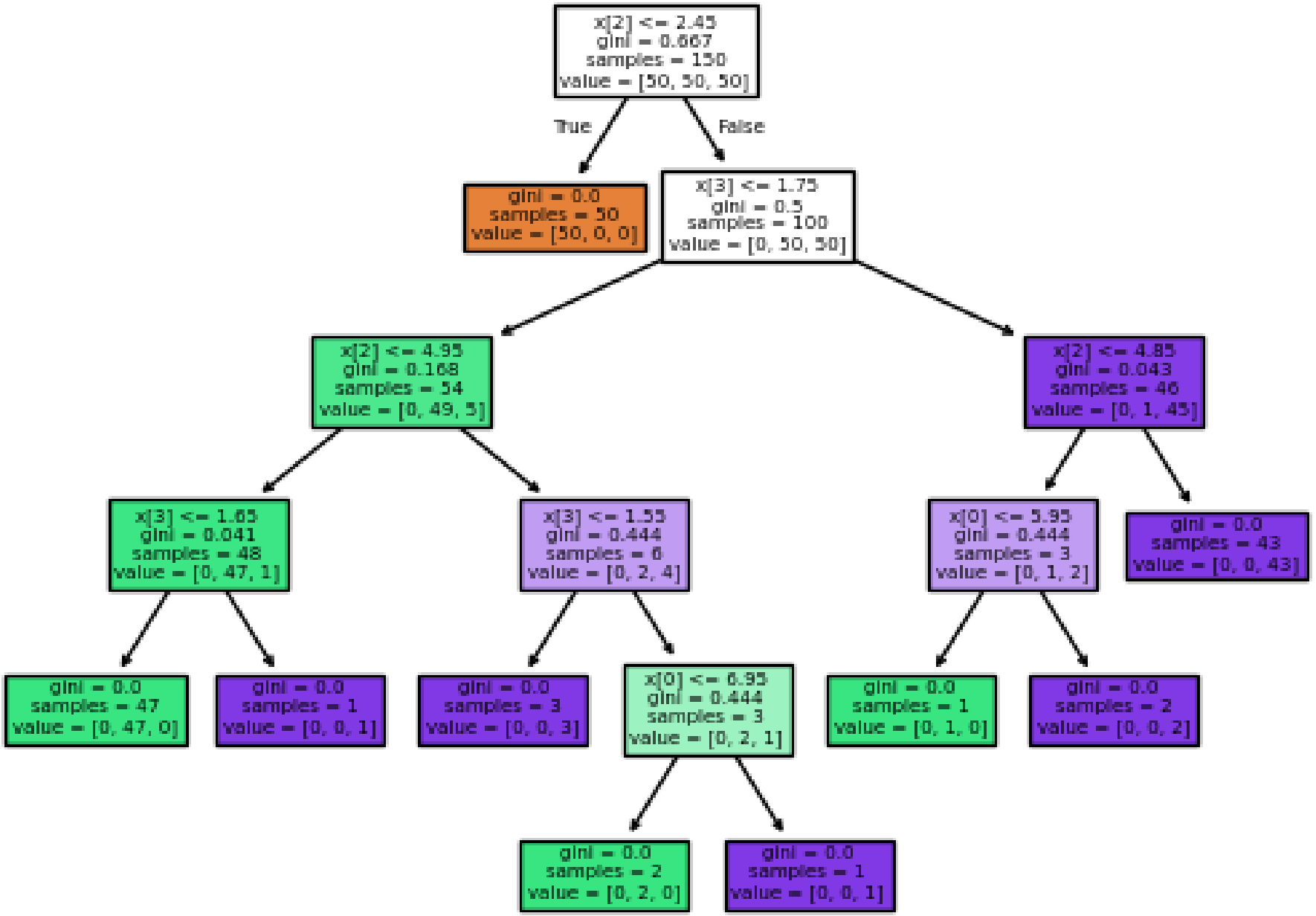

*Iris versicolor*




*Iris virginica*



# Decision tree trained on all the iris features



# Entropy and Information Gain

$$Entropy:$$

$$H(S) = -\sum_i p_i(S)\, log_2\, p_i(S)$$

- **Entropy and Information Gain are the mathematical methods for choosing the best split.**
- Entropy is the level of ‘randomness’ in a distribution.
- Information gain is a measure of the difference in entropy between the set before and after a split.
- The attribute that provides the **highest information gain is chosen as the split attribute**.
- **Gini index** is another measure of impurity or uncertainty.
  It ranges from 0 (completely pure) to 1 (completely impure

$$Information\ Gain:$$

$$IG(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{S} H(S_v)$$

# Gini Impurity

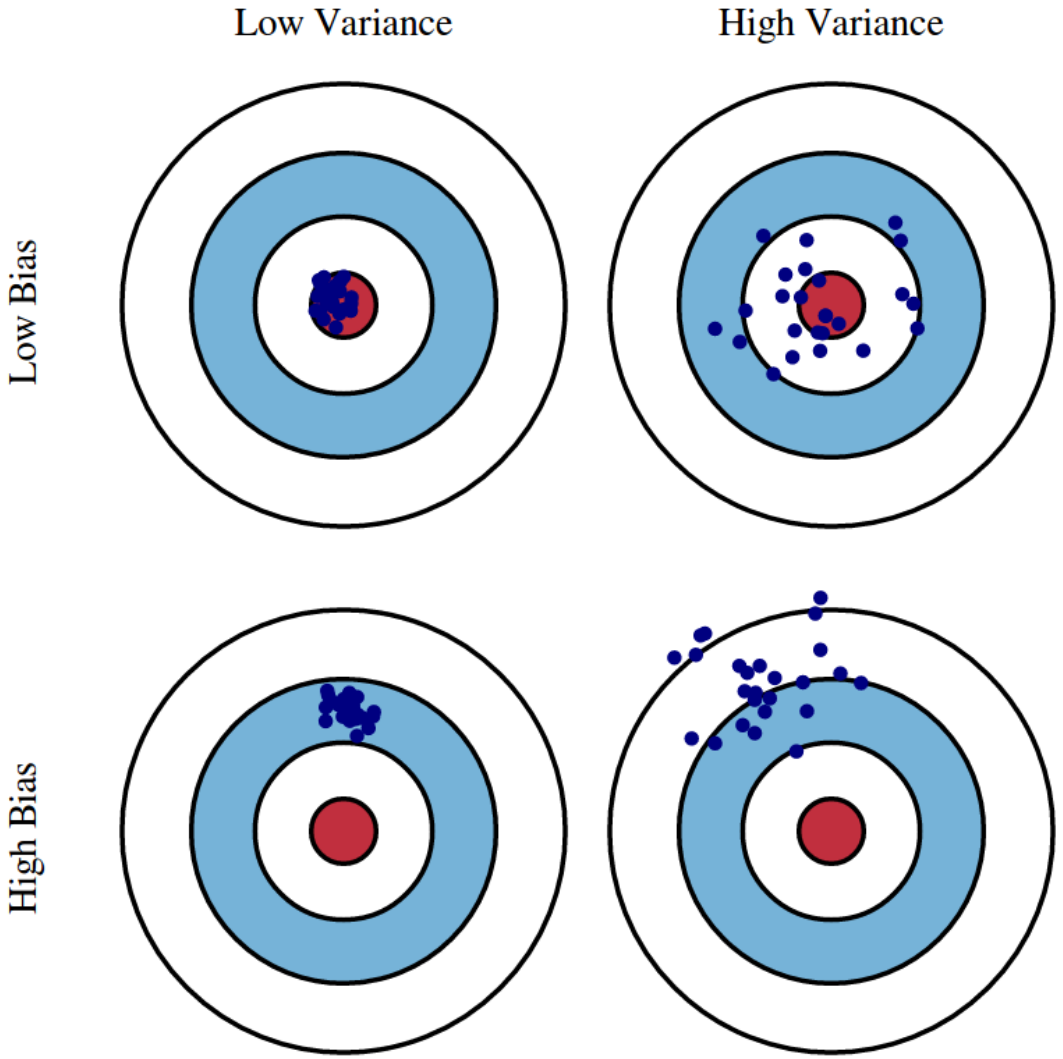
- Gini Index (also known as Gini impurity) is a measure of how mixed (differentiation between classes) or impure a dataset is. Essentially how many classes? If completely pure – all the same class.
- Gini impurity ranges from **0 (completely pure)**, to **1 (completely impure).**
- **Pure = all samples belong to the same class**
- If a node has 50% Class A and 50% Class B, the impurity is high;
- if a node has 90% Class A and 10% Class B, the impurity is lower (or in other words: ‘more pure’).

# Gini vs Entropy

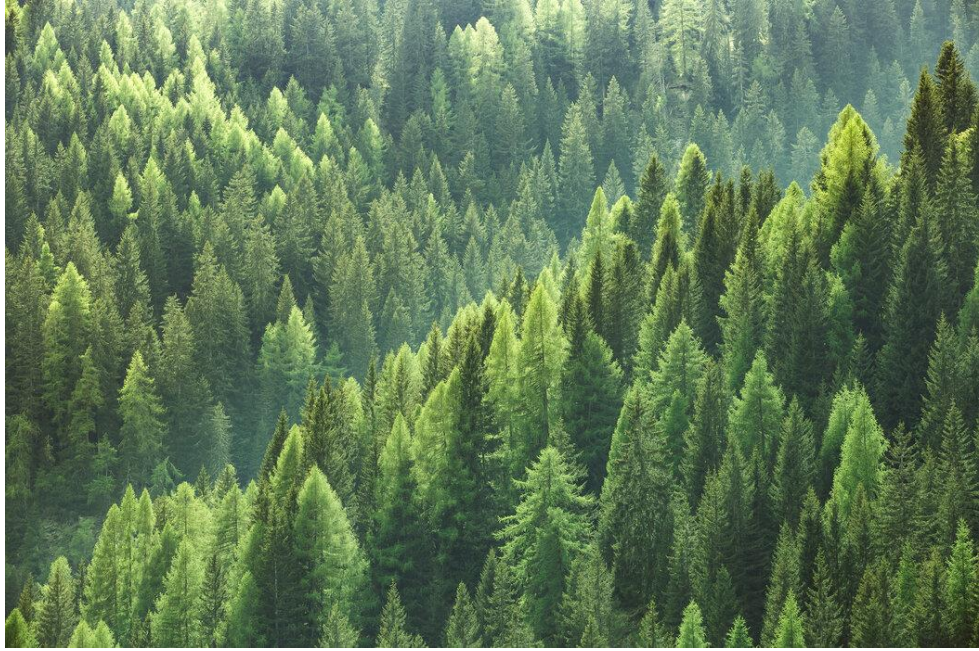
- Gini impurity: A **lower Gini impurity indicates that a split results in better separation of classes** and thus higher "purity" of the resulting nodes.
- Entropy: A **lower entropy indicates that a split results in less uncertainty or disorder**, with more elements belonging to a single class, and thus higher "purity" of the resulting nodes.
- **Both Gini impurity and entropy** can be used as **splitting criteria** in decision tree algorithms, such as CART (Classification and Regression Trees). The choice between them often depends on the specific problem and the preference of the practitioner.
- Empirical studies have shown that decision trees built using Gini impurity and entropy tend to produce similar results in practice.

# Limitations of Decision Trees

- **High Variance** – Small changes in data can lead to very different trees.
- **Bias-Variance Tradeoff** – Shallower trees reduce variance but increase bias, making predictions less accurate.
- **Solution**: **Combine multiple decision trees** to improve stability, accuracy, and generalisation => **Random Forests**



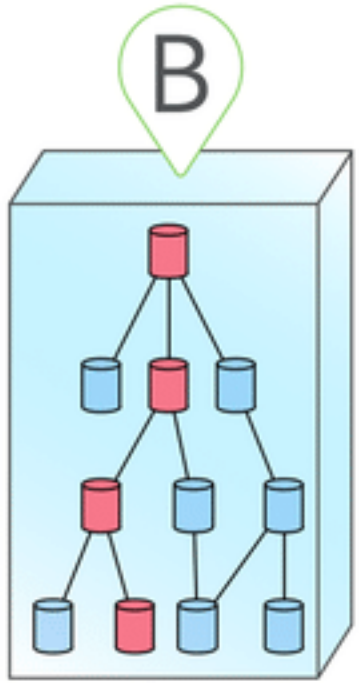
# Random Forests



- To improve performance, we can use **many trees with a random sample of features chosen as the split.**
- Random Forest is a machine learning technique
- **Bootstrap sampling:** each tree is trained on a random subset of data
- **Feature randomisation:** each split considers a **random subset of features**
- **Majority Voting** for **Classification** / **Averaging** for **Regression**
  - As the final prediction is based on multiple trees, it is more robust and generalisable

A

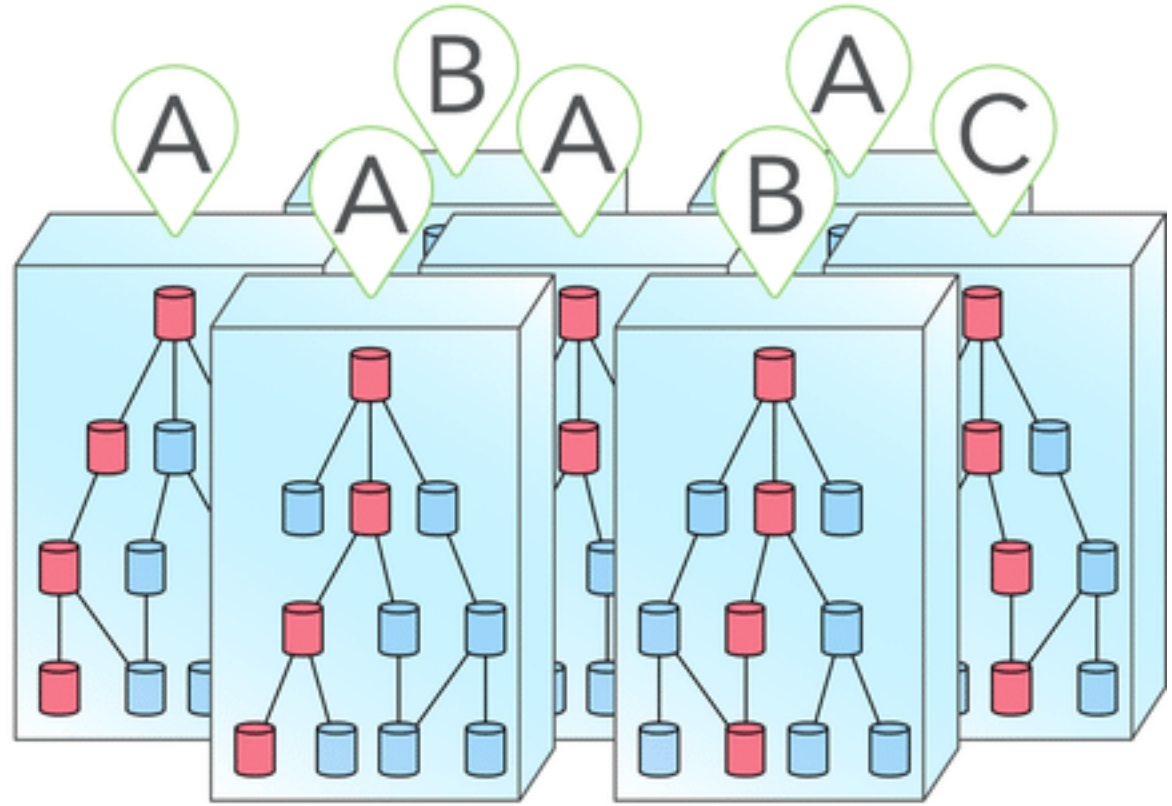
## Decision tree




Single output

B

## Random forest




| A | B | C |
|---|---|---|
| 5 | 2 | 1 |

Majority voting: A

# Why RF usually outperforms a DT

- Suppose there is one very strong feature in the data set. When using ‘bagged’ trees, most of the trees will use that feature as the top split, resulting in an **ensemble of similar trees that are highly correlated**.
- Averaging highly correlated quantities does not significantly reduce the variance.
- By randomly leaving out candidate features from each split, **Random Forests “decorrelates”** the trees (and forces diversity),
- The **averaging process** can **reduce the variance** of the resulting model.

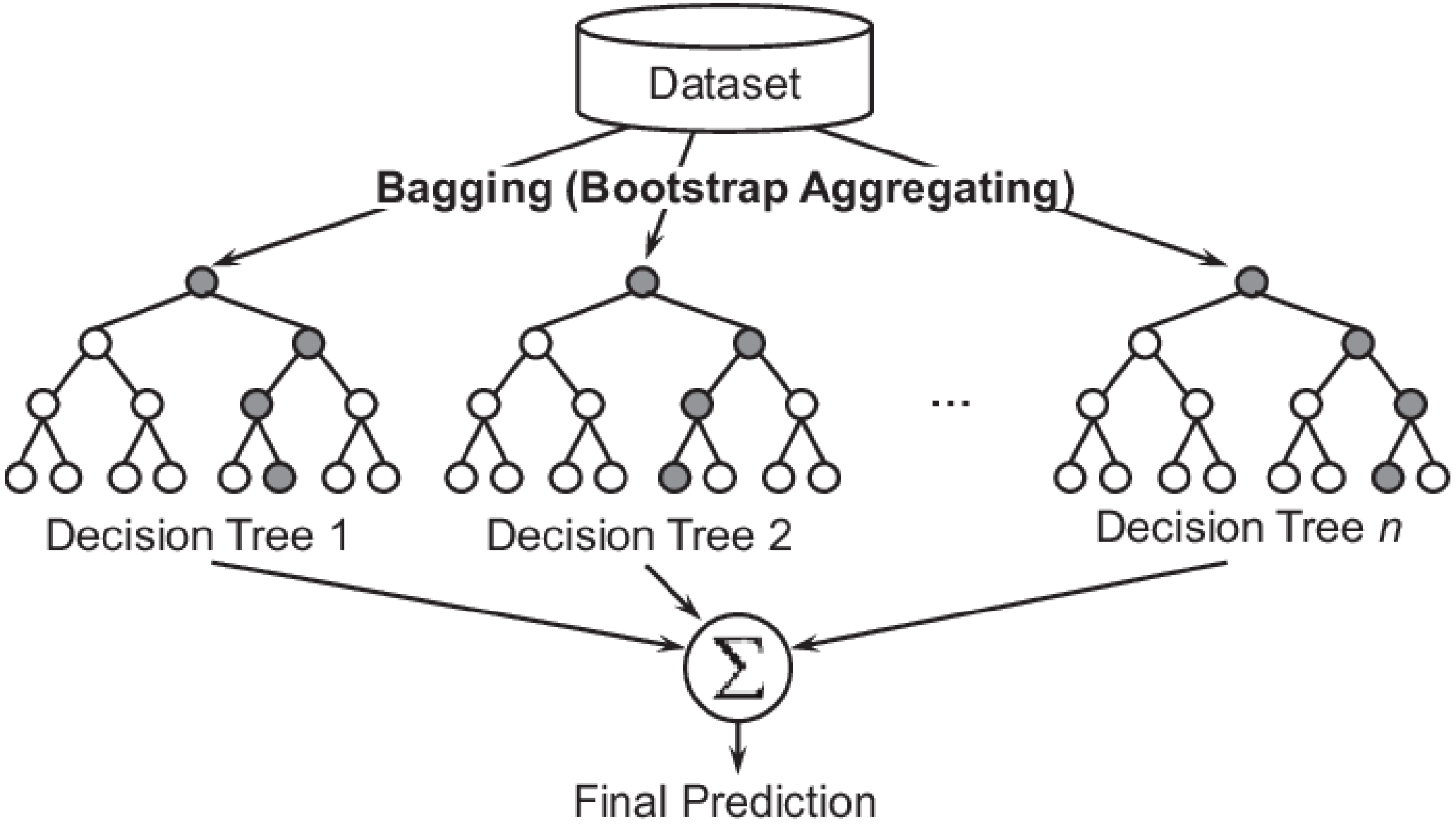
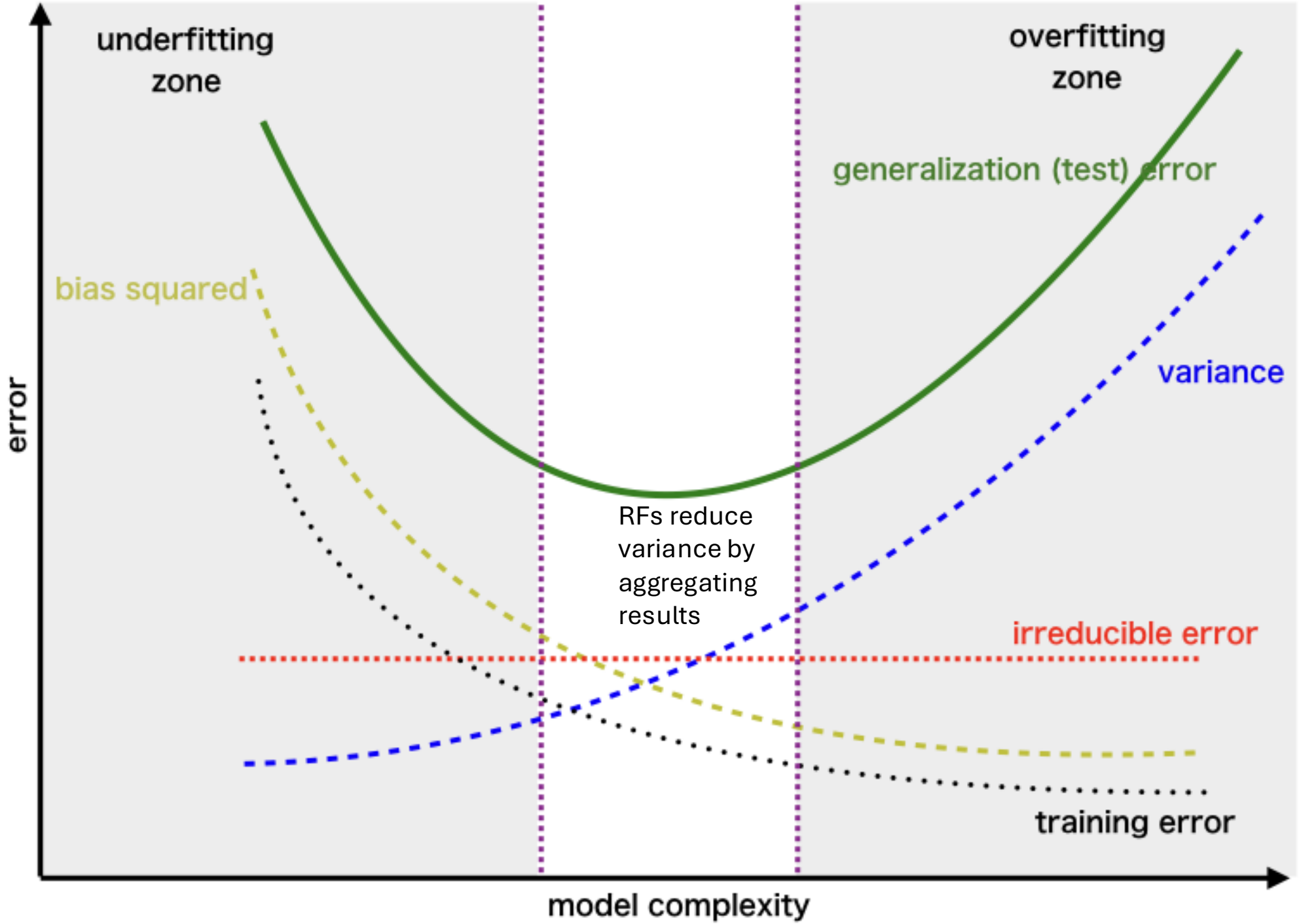



Fig. 1. An Illustration of Random Forest.




DTs can be high variance

# Ensemble methods

- The goal of **ensemble methods** is to combine the predictions of several base estimators built with a given learning algorithm in order to **improve generalisability / robustness** over a single estimator
- **Averaging methods**: Random Forests
- **Boosting methods**: Ada Boost, Gradient Tree Boosting, XGBoost

# Bagging vs Boosting

- Boosting (AdaBoost, Gradient Boosting, XGBoost etc) build **trees sequentially/iteratively** – learning from and correcting previous mistakes
  - This does however make it slower to train… than bagging which can be done in parallel.
- The averaging technique of **Bagging helps to reduce variance** (more stability and less prone to overfitting)
- But **Boosting reduces bias** (better accuracy as a result).
- Whilst bagging techniques (RFs) are less complex/easier to tune, **boosting techniques are more complex** and require **more** careful **hyperparameter tuning**

# Gradient Boosting

Gradient Boosting is an ensemble method that builds decision trees **sequentially**, where each tree **corrects the errors** of the **previous one** using **gradient descent**.


1. Start with an Initial Model
2. Calculate the Residuals (Errors: Difference pred and actual)
3. Fit a New Tree to Predict the Residuals (goal is to build a decision tree that predicts these **errors** instead of the original target)
4. Update Predictions Using **Learning Rate**
5. Repeat Until **Convergence**

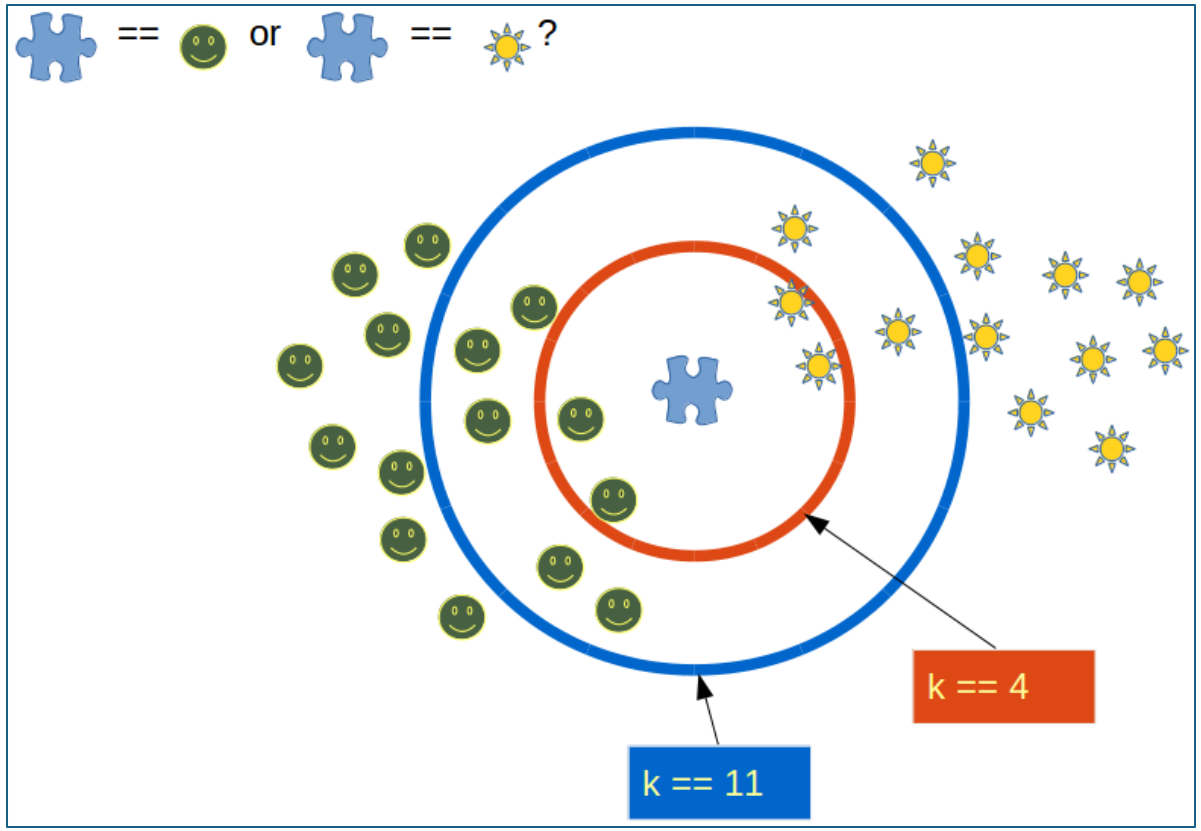
# XGBoost

- XG (Extreme Gradient) improves traditional Gradient Boosting by:
- **Parallel Processing**:
  - Traditional Gradient Boosting builds **one tree at a time**, while **XGBoost builds trees in parallel (by pre-sorting data and efficiently finding splits)**.
- **Regularisation**:
  - Traditional GB doesn't have built-in **regularisation**, but XGBoost includes **Lasso (L1) and Ridge (L2) penalties** to reduce overfitting.
- **Optimised Tree Growth:**
  - Traditional GB **grows trees greedily** (keeps adding branches until a stopping condition) but XGBoost uses **pruning** (removing splits that don't improve performance).

```python
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X, y = data.data, data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train XGBoost model
model = xgb.XGBClassifier(n_estimators=100,
                          learning_rate=0.1,
                          max_depth=3,
                          use_label_encoder=False,
                          eval_metric='logloss')
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.4f}')
```

# Others: Nearest Neighbours Classification

- a type of *instance-based learning* or *non-generalizing learning*
- Classification is computed from a simple majority vote of the nearest neighbours of each point
- KNeighborsClassifier
- RadiusNeighborsClassifier
- The closest instances are determined using some distance measure:
  - Euclidean distance
  - Manhattan distance
  - Minkowski distance




source: https://python-course.eu/machine-learning/k-nearest-neighbor-classifier-in-python.php

# Others: Naive Bayes

- Naive Bayes methods are a set of supervised learning algorithms based on applying Bayes' theorem with the "naive" assumption of conditional independence between every pair of feature records given the value of the class variable.

$$P(y|x_1,\dots,x_n) = \frac{P(y)P(x_1,\dots,x_n|y)}{P(x_1,\dots,x_n)} \approx \frac{P(y)\prod_{i=1}^{m} P(x_i|y)}{P(x_1,\dots,x_n)} \propto P(y)\prod_{i=1}^{m} P(x_i|y)$$

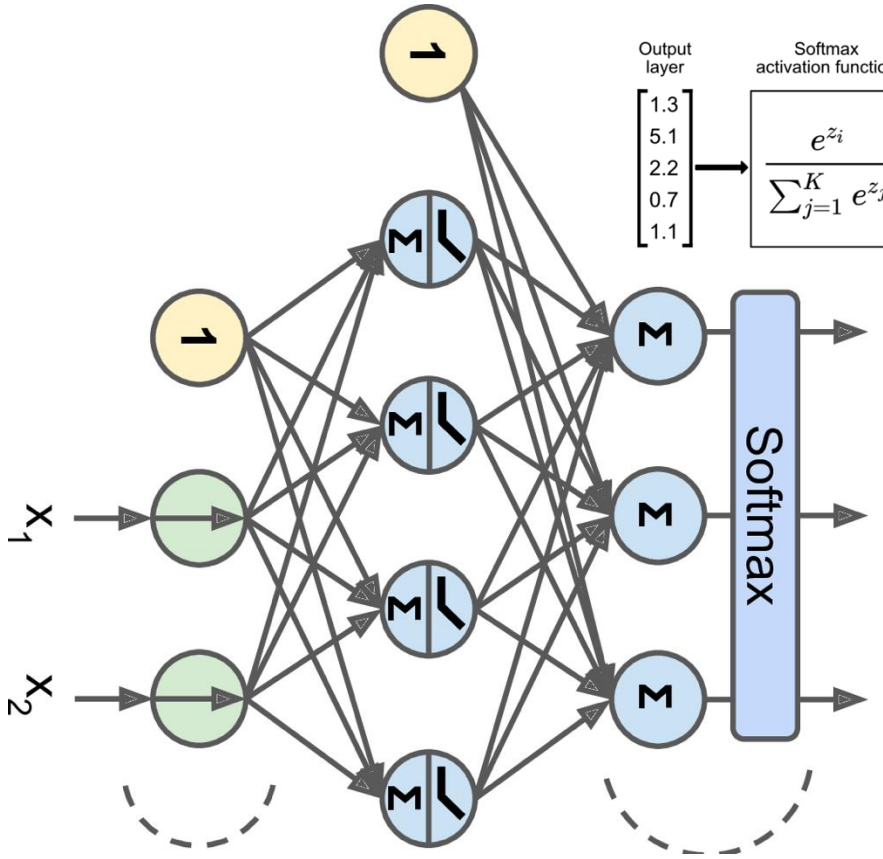
- So it follows that the most likely prediction for $y$ is as follow:

$$\hat{y} = arg\max_{y} P(y)\prod_{i=1}^{N} P(x_i|y)$$

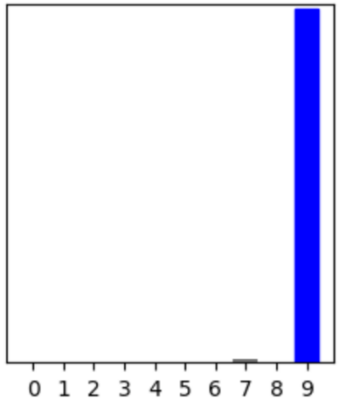
- The different naive Bayes classifiers differ mainly by the assumptions they make regarding the distribution of $P(x_i|y)$: Gaussian, Multinomial, Bernoulli…

# Still to come: Neural Networks for classification

Input




argmax




Ankle boot 99% (Ankle boot)

Hidden layer(s)

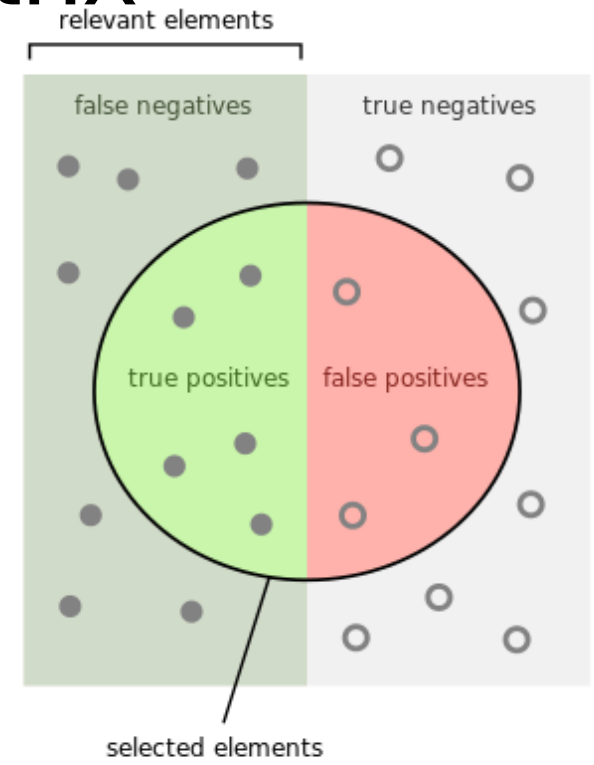
# Performance Metrics: Confusion Matrix

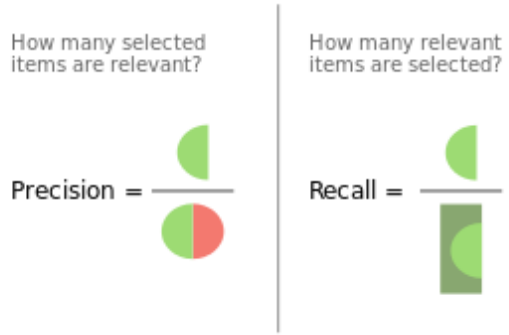
$$\boldsymbol{accuracy} = \frac{correct\ predictions}{total\ predictions} = \frac{\boldsymbol{TP + TN}}{TP + TN + FP + FN}$$

$$\boldsymbol{precision} = \frac{TP}{TP + \boldsymbol{FP}}$$

$$\boldsymbol{recall} = \frac{TP}{TP + \boldsymbol{FN}}$$

$$\boldsymbol{F1} = 2\,\frac{precision\ \times\ recall}{precision + recall}$$

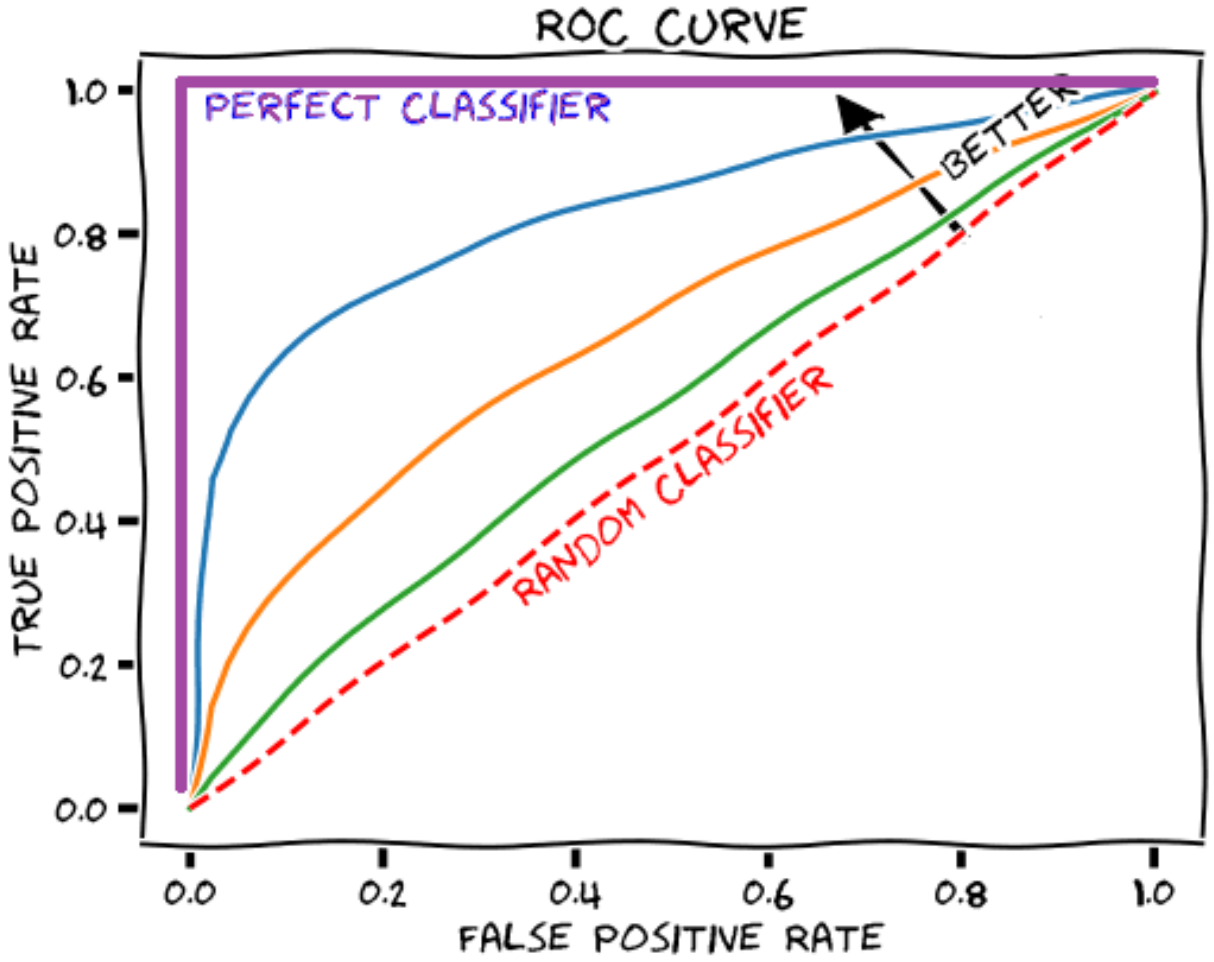


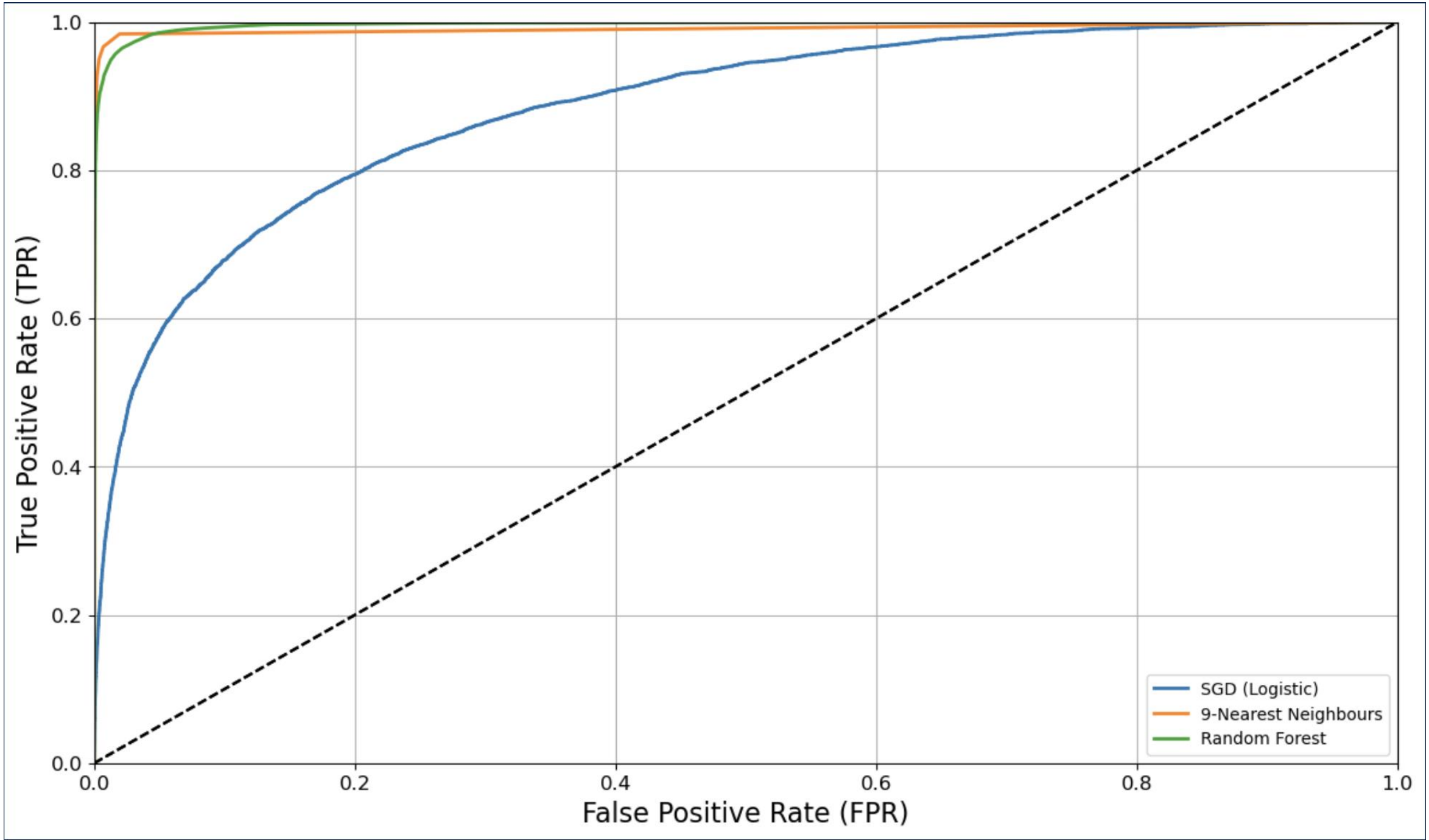
# Performance Metrics: Area under the ROC curve

$$TPR = recall = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$




https://medium.datadriveninvestor.com/understanding-auc-roc-clearly-explained-74c53d292a02


True Positive Rate (TPR)
False Positive Rate (FPR)
0.0
0.2
0.4
0.6
0.8
1.0
SGD (Logistic)
9-Nearest Neighbours
Random Forest

# DT and RF with Sklearn

# Decision Tree Classifier

- from sklearn.tree import DecisionTreeClassifier
- dtree = DecisionTreeClassifier()
- dtree.fit(X_train,y_train)

```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
            max_features=None, max_leaf_nodes=None, min_samples_leaf=1,
            min_samples_split=2, min_weight_fraction_leaf=0.0,
            presort=False, random_state=None, splitter='best')
```

# Decision Tree Classifier

- from sklearn.tree import DecisionTreeClassifier
- dtree = DecisionTreeClassifier()
- dtree.fit(X_train,y_train)
- predictions = dtree.predict(X_test)

# Decision Tree Classifier

- from sklearn.metrics import classification_report,confusion_matrix
- print(classification_report(y_test, predictions))

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.85 | 0.82 | 0.84 | 2431 |
| 1 | 0.19 | 0.23 | 0.20 | 443 |
| avg / total | 0.75 | 0.73 | 0.74 | 2874 |

# Decision Tree Classifier

- from sklearn.metrics import classification_report,confusion_matrix
- print(confusion_matrix(y_test, predictions))

```
[[1995  436]
 [ 343  100]]
```

# Random Forest Classifier

- from sklearn.ensemble import RandomForestClassifier
- rfc = RandomForestClassifier(n_estimators=600)
- rfc.fit(X_train, y_train)

```
RandomForestClassifier(bootstrap=True, class_weight=None, criterion='gini',
            max_depth=None, max_features='auto', max_leaf_nodes=None,
            min_samples_leaf=1, min_samples_split=2,
            min_weight_fraction_leaf=0.0, n_estimators=600, n_jobs=1,
            oob_score=False, random_state=None, verbose=0,
            warm_start=False)
```

# Random Forest Classifiers

- from sklearn.ensemble import RandomForestClassifier
- rfc = RandomForestClassifier(n_estimators=600)
- rfc.fit(X_train, y_train)
- predictions = rfc.predict(X_test)
- from sklearn.metrics import classification_report,confusion_matrix
- print(classification_report(y_test,predictions))

# Classification Report for RFC

- from sklearn.metrics import classification_report,confusion_matrix
- print(classification_report(y_test, predictions))

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.85 | 1.00 | 0.92 | 2431 |
| 1 | 0.57 | 0.03 | 0.05 | 443 |
| avg / total | 0.81 | 0.85 | 0.78 | 2874 |

# Confusion matrix for RFC

- from sklearn.metrics import classification_report,confusion_matrix
- print(confusion_matrix(y_test, predictions))

```
[[2422    9]
 [ 431   12]]
```

# Confusion matrix for RFC

- DTC:

```
[[1995  436]
 [ 343  100]]
```

RFC:

```
[[2422    9]
 [ 431   12]]
```